

3DS "SYSPLUGINS" PRESENTATION

System Plugins

THE SETTING

Most people who still own a Nintendo 3DS has modded it by now.

"Luma3DS" is the Custom Firmware that everyone must install in order for modding to become possible.



Luma3DS

File: 'boot.firm'

or



"FORK" – a clone with added features



Nexus3DS

File: 'boot.firm'

FORKS



Luma3DS



Nexus3DS

All are forks

NOT all forks are equal. While 'Nexus' adds many features, 'PowerPrevent' is 1 tiny addition that changes it so that you have to hold the Start button + Power button to turn the console off. People like this because it's easy to accidentally power off. Yet this alone is an entire replacement for Luma3DS.

But you can only use 1 entire replacement at a time! My Play Coins mod requires elevated always-active background code, and that is only possible through modifying Luma3DS, making my one-feature fork incompatible with the other one-feature PowerPrevent fork.



PowerPrevent_SysModule

This project is a modified version of the Luma3DS Rosalina menu SysModules with an added power mechanic. To prevent accidental pressing of the power button during gameplay, it is now required to hold the START button while pressing the power button in order to shutdown the console.

All credit goes the to AuroraWright and her team who built the fantastic [Luma3DS](#) CFW.

This situation also discourages people from making mods that require background running code, as people often won't choose to replace the CFW with the whole thing again plus one feature.

THE SOLUTION?

We want a way for these features to become one self-contained file, so that:

- 1) The release contains just the feature as a small file, not including the whole Luma3DS
- 2) We can load multiple features compatible with one another



...But Luma3DS does not make it a feature to allow such a thing.
It has a feature to load plugins for games, but not for itself.

SO THEN HOW??

We must modify Luma3DS to make this a feature.

But this is back to the same issue we aimed to solve – having to make a fork.

Luma3DS has no plans to update further, and if I just made my own fork, who would use it? – That's why I have teamed up with Nexus3DS devs, who have already made a fork with some reputation.

That gives us a base to make this project feasible.



Nexus3DS

BUT ACTUALLY HOW?

Luma3DS is comprised of multiple "sysmodules" but the only two that really matter the most for our purposes is **Loader** and **Rosalina**.

- **Loader**: This is what literally loads every 3DS app into memory. This has the earliest possible point to modify any app before it runs.
- **Rosalina**: This actively runs in the background throughout the entire time the 3DS is turned on. For background running code, this is the perfect module to edit, and this is the module most forks make changes to.
- **BOTH** of these can modify any other process, making them very powerful.

What we need done is to make it so that the very FIRST thing these sysmodules do, is run new code to:

- Load self-contained plugin files into memory, then have execution jump to that location in memory and run it.
- Then the first thing each plugin code does once jumped to is setup 'hooks' into the sysmodule. **Without any hooks, nothing will change**, the original code has no reason to ever execute ours.
- Once this setup ends, the current code execution returns to the plugin loading code, to setup the next plugin or finish.

Loader:

- Loads data from cartridge/SD/internal storage into memory



Rosalina:

- Always runs with active threads in the background



BUT WHAT IS MEMORY? HOW TO CHANGE IT?

The 3DS has a couple of useful commands that my code uses. Some of which are:

"FSUSER_OpenFile" -> "Please open this file from the SD card"

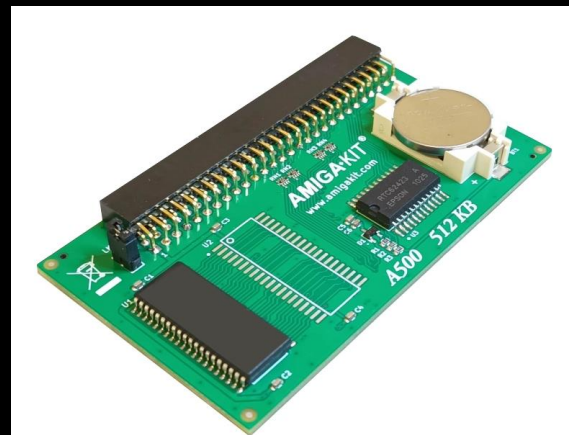
"svcQueryMemory" -> "Is this memory location free?"

"svcControlMemory" -> "Make that free page of memory reserved and writeable for me"

Code never runs directly from a file on the SD card itself. It is always first loaded into RAM, the system runtime memory. "Runtime" meaning, as the system runs, because **memory is completely cleared once the system is off**, and it is a blank clean slate that gets filled from the moment the system turns on.



- Permanent storage
- Survives reboot
- Files, like essay.txt



- Not permanent
- Is constantly updated
- Wiped on reboot
- No file names, just raw data

WHAT DOES CODE LOOK LIKE?

Writing code is a whole lot different to what that code really gets compiled to.
'Compiling' takes human readable code and turns it into a series of instructions.

Writing code usually
looks more like:

```
int a = 5;
int b = 5;
int result;

if (a == b)
{
    result = 1;
    result = result + 10;
}
else
{
    result = 0;
}
```

What it really gets compiled to (and how it behaves in memory):

```
0x01F0  MOV  R0, #5
0x01F4  MOV  R1, #5
0x01F8  CMP   R0, R1
0x01FC  BEQ   0x0500
0x0200  MOV  R2, #0
0x0204  B     0x0510
```

...

```
0x0500  MOV  R2, #1
0x0504  ADD  R2, R2, #10
```

A register representing 'result' gets set to 1,
then has 10 added on to itself

"Pointers" are very important! If you modify this code so that
execution jumped to offset 0x600 instead of offset 0x500, that
most likely would simply crash the system

Execution jumps to offset
0x500 as a result of 5=5

Gets skipped

WHAT ARE HOOKS?

ORIGINAL CODE	
0x0010A0	LDR R0, [R4,#20]
0x0010A4	ADD R0, R0, #1
0x0010A8	STR R0, [R4,#20]
0x0010AC	CMP R0, #10
0x0010B0	BNE 0x0010C0

These are ARM processor instructions.
Let's hook the beginning of them:

PATCHED WITH HOOK	
0x0010A0	LDR PC, [PC,#-4]
0x0010A4	.word 0x00F00000
0x0010A8	STR R0, [R4,#20]
0x0010AC	CMP R0, #10
0x0010B0	BNE 0x0010C0

This is the smallest possible hook. It means:

- 'PC' is the current address being executed, but in arithmetic, it actually refers to current address + 8 (for some reason)
- LDR PC = set PC (currently executed address) to:
- [PC, #-4] = data at (current instruction + 8) - 4 = 0xF00000

- Putting custom code into memory is not enough on its own to affect anything.
- The only way it'll ever be executed is if you hijack existing code to make it jump to yours.
- That jump is called a 'hook' (which reroutes to our 'trampoline').
- Placing a hook requires replacing existing instructions, but those instructions must still run as if your hook never existed.

HOOK / TRAMPOLINE	
0x00F00000	PUSH {R0-R3, LR}
0x00F00004	BL MyCustomCode
0x00F00008	POP {R0-R3, LR}
0x00F0000C	LDR R0, [R4,#20]
0x00F00010	ADD R0, R0, #1
0x00F00014	B 0x0010A8

Not all code is as easy to hook, but this is an example of a solid easy place to hijack and run all the custom code we want before returning

Restored instructions that we had replaced

This is the basis of modding any compiled code, and exactly what our plugins must set up from their Main() code.

WHAT DOES MEMORY LOOK LIKE?

Every file is just a series of bytes. Each byte is essentially just a number from 0 to 255 (256 values, 2^8) or '0x00 to 0xFF' in base 16.

Memory is just that too, just data. All code, or files, is all just data once it's in memory.

That's what makes modifying memory so tricky. If your code 'points' execution to the wrong address, such as 0x800 instead of 0x560, and at 0x800 is data like "hello", the system will try to execute that data as if it were code and instantly crash.

But **not all code and data references are absolute pointers, they can be relative instead**. Code at 0x500, instead of saying "jump to 0x510" may instead say "jump to the current location + 0x10".

1 byte, value 0xF8 aka 248 aka 11111000

Memory View: ☒ 8-bit ☐ 16-bit ☐ 32-bit ☐ 32-bit BE Bit: 7 6 5 4 3 2 1 0
0 0 0 0 0 0 0 0

0x00fdb0	f8	1b	00	00	64	0b	00	00	00	00	00	00	bf	23	00	00	d.....#..
0x00fdc0	02	00	00	b8	7f	eb	00	00	10	00	00	00	0d	00	00	00	[].....
0x00fdd0	00	00	00	00	ff	ff	00	00	86	1b	01	00	b8	4b	01	00	K..
0x00fde0	24	4b	01	00	d0	49	01	00	01	00	ae	46	01	00	e2	2c	\$K...I...F...,
0x00fdf0	00	00	7c	6d	00	00	20	d7	00	00	f0	3a	00	00	94	03	.. m...:....
0x00fe00	00	00	00	00	5f	00	00	00	00	00	00	00	00	00	b5	03	
0x00fe10	00	00	00	03	1e	00	00	00	00	00	00	00	00	00	00	01	
0x00fe20	00	00	00	00	27	01	00	00	00	00	00	00	00	00	00	00	
0x00fe30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00fe40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00fe50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	7f	eb	[]
0x00fe60	e0	1b	3e	00	bc	21	7a	00	e4	23	aa	00	20	24	be	00	...>..!z..#..\$..
0x00fe70	40	37	7c	00	60	01	38	00	00	70	08	01	48	47	54	01	@7 ..8..p..HGT
0x00fe80	20	24	be	00	72	97	60	00	ac	3f	f6	ff	42	5f	f1	ff	\$.r.`..?..B_..
0x00fe90	6e	9e	e7	ff	9a	dd	dd	ff	e0	1b	3e	00	e0	1b	3e	00	n.....>...>
0x00fea0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00feb0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00fec0	00	01	00	01	00	01	00	00	00	00	00	00	00	00	00	00	
0x00fed0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00fee0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00fef0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00ff00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0x00ff10	00	00	00	00	00	03	00	00	00	00	00	00	0e	00	00	00	

(This is random data I dunno what this is either)

WE CAN JUST EXTEND THE CODE, RIGHT?

The idea of plugins is to create a feature in a program that wasn't there before, without becoming a permanent part of the program it modifies. So of course, they must extend the code. This is where understanding memory layout becomes important:

1 page = 0x1000 bytes

0x10000	X	CODE
0x11000	X	CODE
0x12000	X	CODE
0x13000	X	CODE
0x14000	X	CODE
0x15000	RW	DATA
0x16000	RW	DATA
0x17000	RW	DATA
0x18000		

3DS memory works in **pages**, where one page is 0x1000 bytes, this is equivalent to a file size listed as "4KB", 4096 bytes.

X means that page is set to 'executable' - the system will crash if we attempt to write there.

R = Read, **W** = Write, **RW** = Allow both.

Attempting to execute a **R**, **W**, **RW** page will crash.

A page cannot be set to **RWX**, which would allow everything.

-- *BUT we can just set it to **RW**, write code there, then set to **X**.*

After inserting a page at 0x15000, all pointers with the address of 0x15000 and above would all become incorrect – and crash

But plugins load separately to the compiled program, and plugins can be loaded in any order. Where does plugin code go, and how do plugin pointers work?

SO WHERE DOES OUR CODE GO?

Since we can't INSERT code



0x10000	X	CODE
0x11000	X	CODE
0x12000	X	CODE
0x13000	X	CODE
0x14000	X	CODE
0x15000	X	NEW CODE
0x16000	RW	DATA
0x17000	RW	DATA
0x18000	RW	DATA
0x19000		

ALL WRONG

We must put it somewhere ELSE



0x14000	X	CODE
0x15000	RW	DATA
0x16000	RW	DATA
0x17000	RW	DATA
0x18000		
...		
0x60000	X	PLUGIN CODE
0x61000	RW	PLUGIN DATA
0x62000		

Found free memory and put it here without interrupting anything else

Original code and data stays in place

But I mentioned that plugins can go in any order. This means 0x60000 as the new location is not fixed, or something we can know in advance. So how do we solve plugin pointers??

We have two problems to solve.

- 1) We must relocate all pointers to match the new location AFTER loading
- 2) Some pointers will refer to the ORIGINAL program code, but with a RELATIVE address ("Current location - 0x2000" instead of "0x16000")

THE COMPILER ASSUMES CODE STAYS IN ONE PLACE

The base address is assumed to be known already.
Most 3DS processes choose "0x100000".
But how can multiple process occupy the same address?
Well that's where 'virtual address' comes in, but that's a different topic. The short answer is that it's not ACTUALLY at 0x100000 on the physical RAM, but the code will act like it is.
This is not something we need to worry about.

KERNEL / MMU MAPPING TO REAL PHYSICAL RAM

Process A virtual	0x100000	→	physical	0x18100000
Process A virtual	0x101000	→	physical	0x18101000
Process A virtual	0x102000	→	physical	0x18102000
Process B virtual	0x100000	→	physical	0x18340000
Process B virtual	0x101000	→	physical	0x18341000
Process B virtual	0x102000	→	physical	0x18342000
Process C virtual	0x100000	→	physical	0x18580000
Process C virtual	0x101000	→	physical	0x18581000
Process C virtual	0x102000	→	physical	0x18582000

NORMAL PROGRAM LAYOUT THE COMPILER ASSUMES

0x100000	MAIN PROGRAM CODE MAIN PROGRAM CODE MAIN PROGRAM DATA MAIN PROGRAM DATA PLUGIN CODE
0x101000	
0x102000	
0x103000	
0x104000	
0x105000	

VS the reality

0x602000	PLUGIN CODE
0x603000	

THE SAME COMPILED LOGIC DOES:
current location - 0x2000
0x602000 - 0x2000 = 0x600000

So we DO need to worry about:

The compiler builds the code with relative addresses (expecting $0x104000 - 0x2000 = 0x102000$), but since we must relocate plugins in memory, these relative addresses don't point to the correct data anymore. Absolute addresses are fine, as "point directly to 0x102000" remains correct no matter what.

*these addresses are examples, plugin code may end up wherever there is free space

WE MUST TRICK THE COMPILER

In the code we have what are called macros, each macro is a #define, for each plugin section:

```
#define PLUGIN_CODE(id)    __attribute__((section(".plugin_" #id), used))
#define PLUGIN_MAIN(id)   __attribute__((section(".plugin_" #id "_entry"), used))
#define PLUGIN_RODATA(id) __attribute__((section(".pluginrodata_" #id), used))
#define PLUGIN_DATA(id)   __attribute__((section(".plugindata_" #id), used))
#define PLUGIN_BSS(id)    __attribute__((section(".pluginbss_" #id), used))
```

And we use these macros to mark all of our plugin code, so that the compiler knows it belongs to its own sections, NOT the usual code sections that the program itself will use. That alone doesn't solve the issue of relative addresses, but this next trick does:

```
PLUGIN_DATA(coin) void *pluginTable_coin[] = {
    (void*)FSUSER_OpenArchive,
    (void*)FSUSER_CloseArchive,
    (void*)FSUSER_OpenFile,
    ...
};

#define COIN_HOST__FSUSER_OpenArchive    ((Result*)(FS_Archive*,FS_ArchiveID,FS_Path))pluginTable_coin[0])
#define COIN_HOST__FSUSER_CloseArchive  ((Result*)(FS_Archive))pluginTable_coin[1])
#define COIN_HOST__FSUSER_OpenFile      ((Result*)(Handle*,FS_Archive,FS_Path,u32,u32))pluginTable_coin[2])

PLUGIN_RODATA(coin) static const char g_coinFilePath[] = "/luma/coins.bin";
PLUGIN_RODATA(coin) static const char g_lumaPath[] = "/luma";
PLUGIN_RODATA(coin) static const char g_gameCoinPath[] = "/gamecoin.dat";
```

We refer to the original program's functions through a table – this trick forces ABSOLUTE addresses over RELATIVE

Then instead of calling FSUSER_...() directly we call it by proxy through COIN_HOST__FSUSER_...()!

ALL plugin data must be marked!

Since plugin code moves together as a whole, relative addresses into ITSELF is totally fine. This is where the problem flips – if the compiler assumes the plugin lives at 0x104000, absolute addresses now point into that 0x104000... range instead of where the plugin got moved to, over at 0x602000!

CREATING A CUSTOM FILE FORMAT

Luma3DS supports loading plugins for games, in the form of a '.3gx' format - "3DS Game eXtension"

This is a custom file format used by the Rosalina module and is the plugin format most popularly used by CTGP-7, the Mario Kart 7 modpack. *Luma can only load one 3gx.*



Our custom file format is a '.3nx' - "3DS Nexus eXtension" ----->
Every piece of data here plays an important role in correctly loading the plugin into memory. The diagram here shows 'File Only' vs 'gets loaded to a X/RW page'.

Quick run through:

Header is how the code knows the location and size of each section of the file.

Plugin Main, Code, Read-Only Data, RW Data, are all the same macros shown on the last slide. **Main** and **Code** are together on **X** pages (executable code) whereas all other data is placed on **RW** pages.

*Read-Only data is typically only on **R** pages, but we don't really care about that.*

As for **Fast Relocation Table** and **Repair Table**, these are the solutions to both "how do we handle absolute pointers into movable plugins?" and "how do plugins survive if Nexus3DS updates, destroying all the absolute pointers into it?"

We haven't gotten to this yet

HEADER	[FILE ONLY]
Module magic (Rosalina / Loader)	
Plugin ID ("coin", "blur", etc.)	
Code size	
Data size	
BSS size	
Fast relocation size	
Repair size	
Plugin ABI	
Expected host environment	
Metadata size	
Fixed size: 0x30 bytes	
FAST RELOCATION TABLE	[FILE ONLY]
Plugin-to-itself pointer fixes	
Plugin-to-plugin pointer fixes	
e.g.	
"at +0x80, point to myself +0x320"	
PLUGIN MAIN	[X]
PLUGIN_coin_Main()	
Always begins at executable offset +0x0	
PLUGIN CODE	[X]
All other PLUGIN_CODE(...) functions	
Hook code / helpers / callbacks / etc.	
PLUGIN RODATA	[RW]
Strings	
const tables	
patch words	
PLUGIN DATA	[RW]
Host pointer table	
initialized writable variables	
REPAIR TABLE	[FILE ONLY]
Exports	
Host-symbol / marker repair records	
Cross-plugin repair records	
Stable 64-bit keys + patch locations	
16-BYTE BODY ALIGNMENT PADDING	[FILE ONLY]
PLUGIN METADATA	[FILE ONLY]
Plugin-owned appended bytes	
Files are packed with no gaps	
metadataSize includes 16-byte zero padding	

HOW IS A .3NX BUILT?



Marking code and data with macros separates plugin code+data from 'host' code+data. 'Host' being the process the plugin aims to target.

```
ROSALINA_PLUGIN_CONFIG=(  
    "blur|blurPLGbase|10|"  
    "coin|coinrosalina|50|blur"  
)
```

```
LOADER_PLUGIN_CONFIG=(  
    "coin|coinloader|50|"  
)
```

Our 'makeplugin.sh' script reads this configuration – here I have 3 plugins configured.

Once this script runs, it sets up the 'linker' script, creating the code that tells the compiler "put PLUGIN_CODE(coin) into the .plugin_coin segment"

```
/* =====  
   === Plugin Sections ===  
   ===== */  
/* pluginstart */  
/* generated by makeplugin.sh */  
  
. = ALIGN(0x1000);  
.plugin_coin :  
{  
    __plugin_coin_start = .;  
    KEEP(*(.plugin_coin_entry))  
    KEEP(*(.plugin_coin))  
    KEEP(*(.plugin_coin.*))  
    __plugin_coin_end = .;  
} : plugin  
  
__plugin_coin_pad_start = .;  
. = ALIGN(0x1000);  
__plugin_coin_pad_end = .;  
  
.pluginrodata_coin :  
{  
    __pluginrodata_coin_start = .;  
    KEEP(*(.pluginrodata_coin))
```

plugin.c + plugin_hooks.c

- makeplugin.sh config
- generated linker script
- GCC builds a big .elf
- create3nx.py extracts plugin sections
- final .3nx

These segments, once compiled by GCC, are snipped right out of the resulting .elf file using some commands.

HOW DOES THE .ELF FILE HELP?

The .elf file contains not only the entire program + plugin code, but also what every line number of source code correlates to as a compiled address, AND even the address of every pointer.

```
14064000 T __plugin_coin_start
14064118 -> 14064c58  PLUGIN_coin_EditPlayCoins
14064fe4 -> 14064eb8  PLUGIN_coin_PatchMenuCallback
1406503c -> 14064c58  PLUGIN_coin_EditPlayCoins
builder@290af67b67b7:/host$ |
```

By running a devkit command we can get:

- The beginning of Coin code: **0x1406400**
- The address **0x14064118** is a pointer to **0x14064c58**

And so on for the other pointers.

Now here's where the **Fast Relocation Table** comes in:

POINTER DELTA		TARGET DELTA	TARGET
+0x118	->	+0xC58	PLUGIN_coin_EditPlayCoins
+0xFE4	->	+0xEB8	PLUGIN_coin_PatchMenuCallback
+0x103C	->	+0xC58	PLUGIN_coin_EditPlayCoins

0x14064118 – **0x1406400** = **0x118** delta.

Now if the plugin ends up at **0x602000**? The loader code goes to **0x602118** and changes value **0x14064c58** to **0x602c58**!



```
# --- simplified create3nx logic ---
elf = load_elf("rosalina.elf")
for plugin in get_configured_plugins():
    # Snip our marked sections out of the ELF
    code   = elf.extract(".plugin_"      + plugin.id)
    rodata = elf.extract(".pluginrodata_" + plugin.id)
    data   = elf.extract(".plugindata_"   + plugin.id)
    # Examine the ELF's pointer / relocation information
    relocations = find_plugin_relocations(elf, plugin)
    self_fixes = make_relocation_table(relocations)
    host_fixes = make_repair_table(relocations)
    # Assemble the custom 3NX file
    output = (
        make_3nx_header(plugin)
        + self_fixes
        + code
        + rodata
        + data
        + host_fixes
    )
    output = pad_to_16_bytes(output)
    save(plugin.name + ".3nx", output)
```

Example script
to extract plugin
from .elf file
(pseudocode)

THE .3NX LOADING PROCESS

LOADER / ROSALINA STARTS

Call SysPluginLoader_Main()

ALLOCATE TEMPORARY WORKSPACE

Space for up to 31 selected plugins

31 × 296-byte records = 9,176 bytes
Page rounded = 3 temporary 4 KB pages

OPEN /luma/plugins/

SCAN NEXT .3nx

- Read filename priority
- Read each 3NX entry inside the file
- Check whether it targets this module
- Read its 4-character plugin ID

If this ID was already found:
keep whichever entry comes FIRST by:

Priority → Filename → Stacked position

Keep at most 31 selected plugin entries

More files left?

YES

NO

Plugins can reference one another, so the loader needs to know **where every selected plugin actually landed in memory** before it can fix their pointers.

LOAD NEXT TO MEMORY

- Re-open its .3nx file
- Work out required memory size
- Find free pages in memory
- Allocate CODE + DATA/BSS as RW
- Copy CODE into memory
- Copy initialized DATA into memory
- Leave BSS zero-initialized

Load worked?

NO

YES

DISCARD
PLUGIN

MARK
LOADED

More plugins?

YES

FIX FAST RELOCATIONS
ID → REAL START ADDRESS

coin → 0x17A00000
exmp → 0x17A04000
etc.

Fix self + cross-plugin pointers
Dependency missing? Skip Main()

MAKE CODE EXECUTABLE

CODE pages: RW → RX
DATA pages: stay RW

Plugin Main() is always at the very beginning. It returns a Boolean value – that means either **True** or **False**.
False = mark to unload.

By returning **False**, it either means it decided not to load based on a condition, or it was a one-shot plugin designed to simply patch something then unload.

This case of 'failure' would not be a coding, pointer, or dependency error.

NEXT PLUGIN IN
PRIORITY ORDER

Did a required
dependency fail?

YES

NO

MARK THIS
PLUGIN
TO UNLOAD

CALL
PLUGIN_Main()

Main returns?

FALSE

TRUE

MARK
UNLOAD

KEEP
LOADED

More plugins?

NO

YES

CLEANUP

Free every plugin marked for unload.
Successful plugins REMAIN in memory.
Free the 3 scratch pages.

RETURN TO SYSMODULE

Hooks / callbacks / threads can now
execute plugin code

SYSPLUGIN ORDER AND PRIORITY

Now that our system can load multiple plugins and allow them to work together, which one runs first?

Files
Base.0.3nx
Coin.5.3nx
Example.10.3nx



LOWER NUMBER = EARLIER
1. Base priority 0
2. Coin priority 5
3. Example priority 10

Plugins can also be stacked too, with the stacking order replacing numbered priority:

PluginPack.5.3nx		
	+-----+-----+	
#1	base	
	+-----+-----+	
#2	coin	
	+-----+-----+	
#3	exmp	
	+-----+-----+	

```
Offset(h) Decoded text
00000000 3NX$base.....
00000010 ....€...d...qæ.Ä
00000020 d0C#.....base
00000030 .....
00000040 . ä.. ä.0*ä...ä
00000640 .....-¥@.«EY4...
00000650 .....>.y'ä.ÖcÄ...
00000660 ....h.òPV.òÄD...
00000670 ....,òü2Ž.ñÇÜ...
00000680 .....
```

+

```
Offset(h) Decoded text
00000000 3NX$coinH...ä...
00000010 ....,...Ä...È»@æ
00000020 ~x.±coin<...X...
00000030 @...æ!..base....
00000040 d"...X...h"...ö...
00001250 Apply.Set Play
00001260 Coins: %d.--&+.i
00001270 *.!ÿ'.....base
00001280 ....-#*öï-..ä...
00001290 ....%C÷.Ä%....
```

Simple concatenation

Base [0]
↑
Coin points to Base
Coin [5]

Provider runs BEFORE
consumer
=
DEPENDENCY

Coin [5]
↓
Coin points to Example
Example [10]

Provider runs AFTER
consumer
=
REFERENCE

Plugins can contain the same base without issue as only 1 of the same ID gets chosen:

PackA.5.3nx	PackB.10.3nx
[base]	[base]
[featA]	[featB]

DEPENDENCY

Provider runs first.
Failure -> consumer is skipped.

REFERENCE

Provider runs later.
Failure -> consumer must handle this itself.

SYMBOLS AND MARKERS FOR HOOKS

How does a plugin describe WHICH part of the Nexus3DS code to place a hook at?

```
extern bool PLUGIN_base_AddMiscMenuItem(void);
extern bool PLUGIN_base_AddMenuColor(const char* name, u32 color);
PLUGIN_DATA(coin) static void* pluginTable_coin[] = {
    (void*)PLUGIN_base_AddMiscMenuItem,
    (void*)PLUGIN_base_AddMenuColor,
    (void*)svcFlushCache,
    (void*)svcInvalidateCache,
};
#define COIN_BASE__AddMiscMenuItem ((bool (*)(void))pluginTable_coin[0])
#define COIN_BASE__AddMenuColor ((bool (*)(const char*, u32))pluginTable_coin[0])
#define COIN_HOST__svcFlushCache ((void (*)(void))pluginTable_coin[1])
#define COIN_HOST__svcInvalidateCache ((void (*)(void))pluginTable_coin[2])
```

Earlier we saw the table method:

- Tables have references to **symbols**
- **Symbols** are named functions or pieces of data
- We can use the .elf to find the compiled addresses they landed at

But **symbols** that refer to code can only refer to a function. What if we don't want to replace a whole function?

Technically we can **check compiled code** and know the **exact address to target** instead, but that requires high maintenance.

Because if Nexus3DS updates, we have no automatic way to find all those hook points again, it takes much more manual effort. *A slight change compiles all addresses differently.* A **PLG_MARKER** gives us a way we can refer to any unnamed line of code. This **//** means it is just a comment, and has zero effect on the compiled code.

*How is the **marker** found after compiling?*

GCC records **what each statement means**, not just how it was typed. The marker script matches the comment to that GCC statement, then the .elf debug info tells us **what address it compiled to**.

```
while(!preTerminationRequested)
{
    svcSleepThread(50 * 1000 * 1000LL);
    if (menuShouldExit)
        continue;

    Cheat_ApplyCheats();

    u32 kHeld = scanHeldKeys(); // PLG_MARKER(powerprevent_marker_key_scan)

    if(((kHeld & menuCombo) == menuCombo) && !g_blockMenuOpen)
    {
        menuEnter();
        if(isN3DS) N3DSMenu_UpdateStatus();
    }
}
```

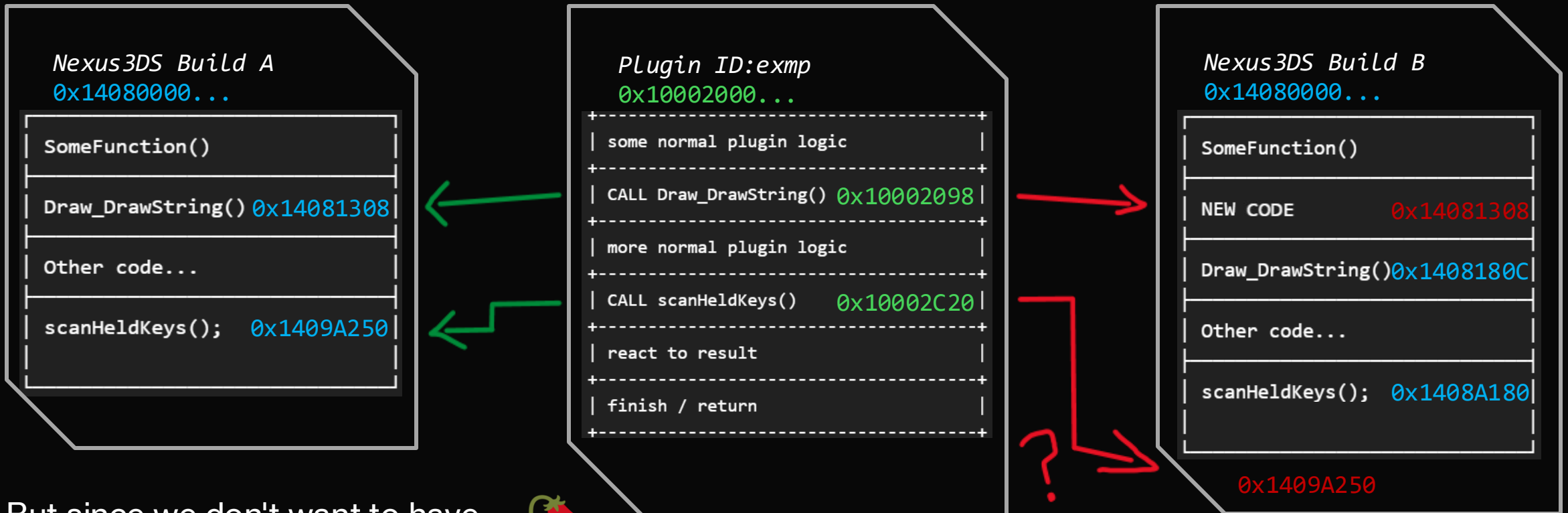
This is code *inside* of a function, rather than only getting the function entry address

This marker method is for pointers into the host only, not cross-plugin refs

BUT WHAT HAPPENS WHEN CODE UPDATES?

So if **modifying code then compiling again shifts every address**, doesn't that break every plugin?
How can plugins accommodate for other plugins, or the host, updating?

If we assume this is never a problem, we now have a **fully complete system**.
Plugins can load, run, point to one another and the host correctly, and rebuilding is easy!



But since we don't want to have a stiff system, we aren't done yet...



How do we find the correct addresses again?

WE CAN HASH THINGS

A hash is a one-way algorithm to turn "hello" or "{the entire dictionary}" into a fixed-size unique* result.

HOW big should the hash be?:

u8	8 bits	256 possible values
u16	16 bits	65,536 possible values
u32	32 bits	4,294,967,296 possible values
u64	64 bits	18,446,744,073,709,551,616 possible values
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^		

The smaller the hash, the higher the chance of *collisions.
This is when two inputs produce the exact same hash.

u64 is only 8 bytes, and has an enormous amount of possibilities, making it suitable to uniquely label every possible **symbol** and **marker**, creating handy look-up tables for our plugins.

Since hashes are one-way, specialized hashing algorithms are commonly used in password databases.

We consistently get the same hash no matter the build

SYMBOL / GCC EXPRESSION	U64 HASH
Draw_DrawString()	0x91A0C4B7E22D1035
svcSleepThread()	0x2E8716C31AFA9072
menuShouldExit	0xB748D11F82965C04
kHeld = scanHeldKeys();	0x62FD7903B418CE21
x = 16;	0xC5138AE45D903761
if (menuShouldExit) {	0x3A9472C0E1885FBD

Our plugin's **Repair Table** has the key, so we ask for the address using that key / hash!

BUILD A produces...

SYMBOL / GCC EXPRESSION	ADDRESS
Draw_DrawString()	0x14081308
svcSleepThread()	0x14027D94
menuShouldExit	0x140F08B0
kHeld = scanHeldKeys();	0x1409A250
x = 16;	0x140A10BC
if (menuShouldExit) {	0x1409A224

BUILD B produces...

SYMBOL / GCC EXPRESSION	ADDRESS
Draw_DrawString()	0x1408180C
svcSleepThread()	0x140282B0
menuShouldExit	0x140F12E4
kHeld = scanHeldKeys();	0x1408A180
x = 16;	0x140A16F8
if (menuShouldExit) {	0x1408A154

U64 HASH	ADDRESS
0x91A0C4B7E22D1035	0x14081308
0x2E8716C31AFA9072	0x14027D94
0xB748D11F82965C04	0x140F08B0
0x62FD7903B418CE21	0x1409A250
0xC5138AE45D903761	0x140A10BC
0x3A9472C0E1885FBD	0x1409A224

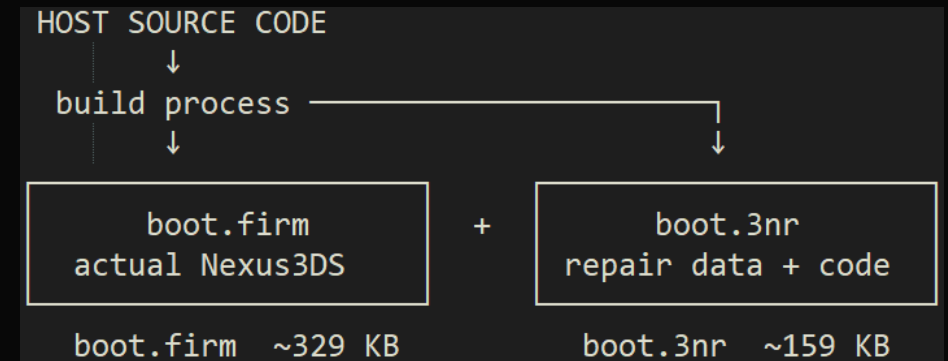
U64 HASH	ADDRESS
0x91A0C4B7E22D1035	0x1408180C
0x2E8716C31AFA9072	0x140282B0
0xB748D11F82965C04	0x140F12E4
0x62FD7903B418CE21	0x1408A180
0xC5138AE45D903761	0x140A16F8
0x3A9472C0E1885FBD	0x1408A154

Final table alongside Nexus3DS

INTRODUCING THE .3NR FILE

boot.3nr is our new companion file built alongside **boot.firm**. The **.3nr** (3DS Nexus Repairer) contains:

- A large lookup table from **u64 key/hash** -> **host address**
- The **repair code** that uses that table to fix outdated pointers



boot.3nr is big enough that it makes more sense as a separate optional companion file. Without it, we skip plugin loading.

HEADER	[FILE ONLY]
"3NR!" magic	
Build-pair ID	
Repair-code offset + size	
Runtime BSS size - [RW]	
Loader table count	
Rosalina table count	
Fixed size: 0x20 bytes	
REPAIR / FIXER CODE	[X]
Main() begins at executable offset +0x0	
Scans plugin Repair Tables	
Looks up hashes in the tables below	
Rewrites outdated plugin pointers / relocations	
~5.2 KB of repair code	
LOADER HOST LOOKUP TABLE	[FILE ONLY]
u64 key/hash -> u32 current address	
u64 key/hash -> u32 current address	
...	
12 bytes per entry	
ROSALINA HOST LOOKUP TABLE	[FILE ONLY]
u64 key/hash -> u32 current address	
u64 key/hash -> u32 current address	
...	
12 bytes per entry	

The **.3nr** code uses the **Repair Table** from a plugin against its own **Look-Up Tables** to fix the pointers in the plugin's **Fast Relocation Table**.

We don't use the repair table every boot however, because it has much more work to do:

- The reloc table already stores where to patch and what offset to use, so loading mostly amounts to simple additions and writes.
- The repair table instead must look up each stable hash in the much larger **.3nr** tables, find its new address, and rewrite the cached values in the **.3nx** file.
- This search means scanning through thousands of hashes per repair.

Exactly WHEN do we use the repair table?

The repair/fixer code gets loaded from the **.3nr** upon a mismatch in **expected environment**.

The code itself & the BSS (zero-initialized RW data), once rounded to pages, **occupies ~20KB of memory**. Since it only runs when necessary, it is more efficient to allocate it temporarily, loaded from the **.3nr**.

So... how do we detect this **expected environment**?

DETECTING AN ENVIRONMENT CHANGE

When a `boot.3nr` file is built, we store the 64-bit '**Build-Pair ID**' which is stored in the matching `boot.firm` so that the plugin loading code can verify the pair `boot.3nr` file.

When a plugin is built, that plugin's **Exports / Look-Up Table** itself is hashed to create the **ABI hash** – a 64-bit fingerprint of the table.

ABI = Application Binary Interface

This **Look-Up Table** is a map of every *hash* -> *address delta* for **symbols** within the plugin, and unlike the `.3nr` which is separate from the `.firm`, this is stored within the plugin itself.

A `.3nx` file contains:

FAST RELOCATION TABLE

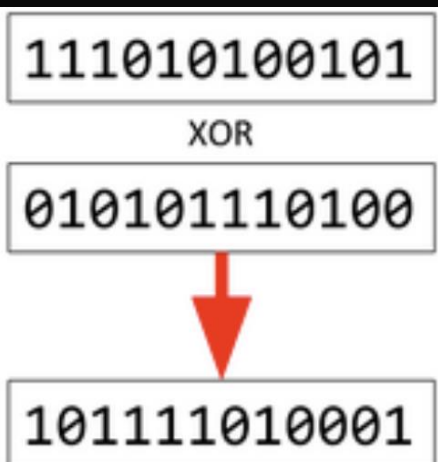
where my pointers currently point

CODE / DATA

REPAIR TABLE

└ Exports	← describes MY public ABI
└ Repair records	← describes pointers I need repaired

XORing is a simple & fast way to combine multiple keys into one result. Any key being changed, added, removed, will change the **XOR** result.



XOR result = **current environment**. This is a very fast operation, and if all plugins agree on the same **expected environment**, we don't have to touch the `.3nr` file or repair tables at all and go straight to fast relocs.

Environment Key =
Build-Pair ID
XOR Plugin A ABI
XOR Plugin B ABI
XOR Plugin C ABI
...

HOST UPDATED	-> Build-Pair ID changes
PLUGIN UPDATED	-> its ABI changes
PLUGIN ADDED	-> another ABI joins the XOR
PLUGIN REMOVED	-> one ABI leaves the XOR

One last thing:

'Fastboot' lets you choose any `boot.firm` to boot. So without having to manually swap the `boot.3nr` file, our code renames `boot.3nr` to `boot.{Build-Pair u32}.3nr`, using this as a fallback for any subsequent boots. And with that, **our system is complete.** 🎉

CONCLUSION

What did we build?

System mods that require elevated permissions and background running code finally have an accessible entry point that doesn't require yet another fork of the whole custom firmware.

Features can now be built, shared, combined and updated independently as **Sysplugins**.

Earlier this year I was pretty surprised to find out that making something to the effect of a custom Sysplugin wasn't a supported feature – so I'm pretty happy to have made it possible!

